litefont

A small font engine for OpenGL and the CPU

Shane Transue

1 Overview

litefont turns OpenType fonts into crisp text. It reads the font file itself, renders hinted glyph images with exact antialiasing, lays out paragraphs and labels, and draws them through plain OpenGL 1.1 or into an image on the CPU. It is written in C++20, depends only on the standard library and the system's OpenGL, and needs no extension loader.

- **Fonts:** TrueType and CFF (`.otf`) outlines, collections (`.ttc`), kerning from GPOS or the kern table, names and traits; damaged files fail cleanly.
- **Families:** regular, bold, italic and bold italic faces, simulated when missing; fallback families for missing code points; a catalog that finds families by name in a folder.
- **Glyphs:** exact-area coverage with vertical hinting; outlines, sharp and blurred shadows, all cached in a texture atlas.
- **Layout:** wrapping, alignment, tabs, letter spacing, line limits with an ellipsis, superscripts and subscripts, underline and strikethrough, per-range colors and styles, anchors and quarter-turn rotation.
- **Editing queries:** caret boxes, selection rectangles, hit tests and caret movement.

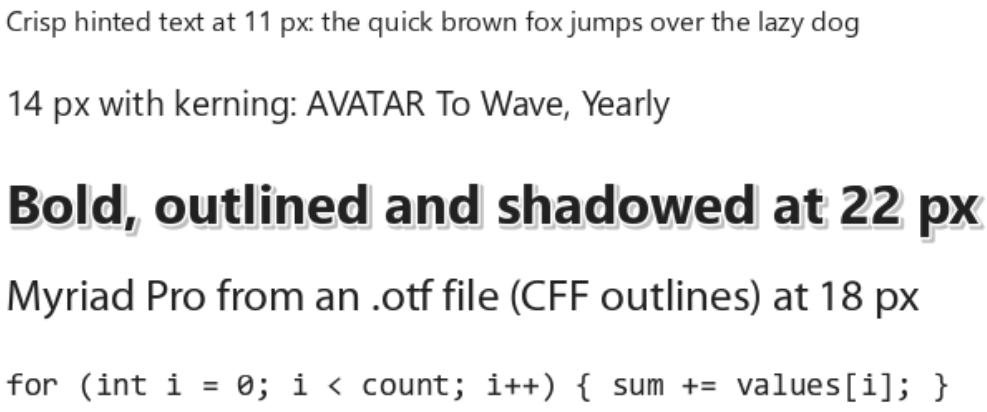


Figure 1: Text drawn by litefont on the CPU at 150 percent scale: hinted small text, kerning, a bold outlined and shadowed label, a CFF font and monospaced code.

2 Quick start

A program declares a font and a renderer, loads the font once, and draws wherever it already draws. The renderer needs no OpenGL context until its first draw.

```
#include <litefont/litefont.h>

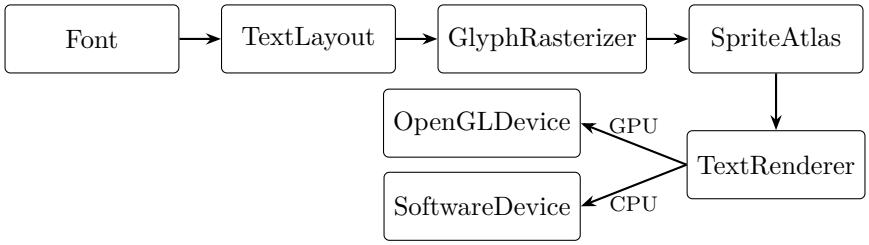
litefont::Font font;                        // (1) declare
litefont::OpenGLRenderer renderer;

void init() {
    font.load("DejaVuSans.ttf");            // (2) load once
}

void display() {
    glBegin(GL_TRIANGLES);                  // the program's own drawing
    // ...
    glEnd();
    renderer.drawText(font, 16.0f, "Hello world", 20.0f, 40.0f);   // (3) draw
}
```

Positions are pixels from the top-left corner of the viewport. `measureText` gives the box a string takes, for centering or right alignment, and `setScale` sets the device pixels per pixel on high-density screens.

3 How it works



A **Font** owns its file's bytes and answers questions about them: metrics, the glyph for a code point, advances, kerning and outlines. **TextLayout** places the glyphs of a **Text** (its content, family, size, colors and layout settings) on whole-pixel baselines. The **GlyphRasterizer** renders each glyph at its size, hinted so baselines and x-heights land on whole pixels, and the **SpriteAtlas** keeps the images in texture pages. The **TextRenderer** turns placed glyphs into quads and hands them to a **GraphicsDevice**: **OpenGLDevice** on the GPU or **SoftwareDevice** on the CPU, which use the same blending and give the same pixels to within one level.

A Text is laid out once and kept. Drawing it again, or moving it, costs no layout and no glyph rendering; only a change to how it looks starts over.

4 Drawing with OpenGL

`OpenGLRenderer` is the immediate-mode front end. Each `drawText` call draws at once, in call order with the program's own drawing. Behind it, every distinct string with its font, size, style and color keeps its layout and glyphs, so a string drawn every frame, even at a new position, costs a lookup and moving its quads. The least recently drawn strings are dropped once the cache holds more than 1024. `draw(text, x, y)` does the same for a Text with all its own settings.

The renderer draws through OpenGL 1.1 alone: the fixed-function calls every desktop OpenGL library exports directly, so it runs in any compatibility context, next to `glBegin` and `glEnd`, without a loader. Glyph pages are alpha textures, modulated by the vertex color and blended with straight alpha. For each draw it sets what text needs:

- an orthographic projection over the viewport;
- blending, texturing, the client arrays, and the scissor for clips;
- depth test, lighting and face culling switched off.

Before drawing it saves the program's state, and afterwards it restores it, with `glPushAttrib`, `glPushClientAttrib` and the matrix stacks. `setRestoreState(false)` skips this for programs that manage their own state. The renderer makes its textures at the first draw; `release()` deletes them while the context is current.

5 Texts and labels

A `Text` holds everything about how a piece of text looks. Changing a setting marks it for a new layout, except the offset and clip, which only move or cut what is drawn.

Setting	Effect
content, family, size	UTF-8 text, the faces it uses, its size in pixels
style, style runs	a face per range of bytes, superscript or subscript, underline, strikethrough
color, color runs	a color per range of bytes, such as syntax colors
outline, shadow	a border around each glyph; an offset shadow, optionally blurred
background	a box behind the text, grown by padding
max width, align, line spacing, tab width	paragraph layout
letter spacing, max lines, ellipsis	spacing between glyphs; cutting long text with an ellipsis
anchor, rotation	the point of the box that lands on the position; a quarter turn
offset, clip	where it is drawn and what it may cover, with no new layout

`TextCursor` answers the geometry an editor needs from a layout: the caret box, selection rectangles, the position a point hits, and where the caret moves by character, word or line.

6 Fonts

A `Font` loads a `.ttf`, `.otf` or one face of a `.ttc` file, from a path or from bytes in memory. A `FontFamily` groups up to four faces and simulates bold and italic where a face is missing; fallback families supply code points the family lacks, and the replacement character marks those nobody has. A `FontCatalog` reads only the names and traits of the fonts in a folder, from a few small reads per file, and loads a family by name. litefont never looks for system fonts by itself: it reads the files and folders a program names.

7 Building

litefont builds with CMake 3.20 or newer and any C++20 compiler.

```
cmake -S . -B build
cmake --build build --config Release
```

`LITEFONT_BUILD_EXAMPLES` adds the freeglut example and a command-line renderer, `LITEFONT_BUILD_TESTS` the test suite, and `LITEFONT_OPENGL` can turn the OpenGL renderer off for CPU-only use. A project adds litefont with `add_subdirectory`, or installs it and uses `find_package(litefont)`; either way it links `litefont::litefont`.